

Chapter 1: Code Implementation

In this project, all numerical experiments and simulations are written in C++/C with the support of basic iterative solver and simple parallel data management provided by PETSc. We perform numerical experiments in quasi-uniform mesh as defined in ... We talk about implementation of a coupled FEM-FVM code without losing generality and flexibility of optimization in the future.

It structures as follows that a general implementation of unstructured mesh will be given, and a more specific example of application regarding our logically rectangular mesh will be discussed in details. All application are expressed in two dimension scenario.

1.1 Geometry

The mesh in the code is defined inspired by the classical viewpoint of Boundary Representation (BREP) Baumgart (1975). In two-dimension case, the element is bounded by a set of edges, and the edges are bounded by a set of vertex. The first and most important aspect of creating a FEM or FVM code is indexing the geometric objects. Assuming the the following representation of global geometric object.

$$\begin{aligned} \text{Vertex} \quad V_i, \quad i \in ID_V, \\ \text{Edge} \quad E_i, \quad i \in ID_E, \\ \text{Cell} \quad C_i, \quad i \in ID_C, \end{aligned} \tag{1.1}$$

where ID_V , ID_E and ID_C are countable set containing all the non repeating global index for vertices, edges and faces respectively. We use superscript to label the subset of global index, e.g., $ID_V^{E_i} \subset ID_V$ represents the global index of boundary vertex associated with the edge E_i , which is a subset of global index set of vertex.

At the same time, assuming the following representation of local geometric

object.

$$\begin{aligned}
\text{Vertex } v_i, \quad i \in id_v, \\
\text{Edge } e_i, \quad i \in id_e, \\
\text{Cell } c_i, \quad i \in id_c
\end{aligned} \tag{1.2}$$

where id_v , id_e and id_c are countable set containing local index for vertices, edges and faces corresponding to a given geometrical object. We use superscript to label the target geometrical object, e.g., $id_v^{E_i} = \{0, 1\}$ represents the local index of boundary vertex associated with the edge E_i .

We connect global index and local index with a one-to-one local to global mapping as

$$f : id \rightarrow ID. \tag{1.3}$$

In implementation, we usually don't have to actually define a local to global mapping explicitly. Consider an array(vector) containing global index, the array index is a natural representation for local index set. With the help of array representation, we can reverse to a global to local mapping within each target geometrical objects.

1.1.1 General Unstructured Mesh

We start with a brief analysis of what information should a target cell should see from its perspective. Each polygon cell E_i should have the following links to other geometric objects as

- Adjacent polygons connected by edges;
- Additional adjacent polygons connected with vertices, but not sharing an edge;
- Indicident vertices;
- Edges that bounds this element.

Within the local set, we can relate edges and vertices directly. For convinence, each edge j should has links to:

- Two polygons that sharing this edge;
- Two vertices that bounds this edge.

In FEM and FVM computation, we work with a orientable mesh, where "clockwise" and "counterclockwise" can be defined consistently. Hence, we can use half edge data structure to regulate direction in a unstructured polygon mesh. In figure 1.1, we

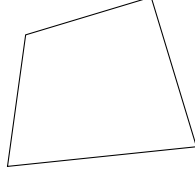


Figure 1.1: An illustration of the half-edge data structure

present an illustration of the half edge data structure applied to a quadrilateral mesh. For each target cell E_i , it sees the vertex V_i and the local link l_i to the next vertex V_i^+ . Hence the direction of the link $l_i : V_i \rightarrow V_i^+$ is determined from the perspective of the target cell C_i .

1.2 Logically Rectangular Mesh

Mesh generation is not the main focus of this project, hence we adopt a much simpler way of implementing quadrilateral mesh, the logically rectangular way. We generate the quadrilateral mesh by randomly distorting each interior vertex coordinates. There is no affine mapping between reference square mesh and generated mesh. The indexing of geometrical objects remain untouched through the vertex distortion.

In our case, all the indices are represented with cartesian coordinates. Therefore, we can setup local to global mapping of geometric objects easily with linear functions. Consider a mesh with $M \times N$ cells, the total number of vertex is $(M + 1) \times (N + 1)$ and the total number of edges is $M \times (N + 1) + (M + 1) \times N$. We count horizontal edges first, and the vertical edges are counted starting from $M \times (N + 1)$.

Before we dig into index convention of a this kind of mesh deeply, we introduce a reshape function communicating N-dimensional data and One-dimensional data.

1.2.1 Reshape N-dimensional Data

The natural representation of a 2D cartesian data is a pair of index (i, j) . However, we usually reshape the 2D data into a 1D array for a more natural representation in computation. We give a more general reshape function that projects N-dimensional data to a one-dimensional data. This reshape function can also be used for tensor data, e.g., base smoothness indicator $??$. There exists a bijective mapping as

$$i \leftrightarrow (i_0, i_1, \dots, i_{N-1}). \quad (1.4)$$

The algorithm 1 illustrates the simple implementation of this reshape function. We

Algorithm 1 N dimensional reshape

Consider a N-dimensional array $\{n_0 \times n_1 \times \dots n_{N-1}\}$.

Let $I = 0$ be the result, and $a = 0$ be the multiplier.

Consider an input N-dimensional index $\{i_0, i_1, \dots, i_{N-1}\}$.

- 1: **for** rank $r = 0, 1, \dots, N - 1$ **do**
 - 2: $I = I + i_r * a$.
 - 3: $a = a \times n_r$.
 - 4: **end for**
-

can also reverse the process reshaping 1D data back into N-dimensional data with modulo and integer division operation. We use these two index interchangeably. We introduce a new notation highlighting the relation between N-dimensional and one-dimensional data as

$$i(i_0, i_1, \dots, i_{N-1})_{n_0, n_1, \dots, n_{N-1}}, \quad (1.5)$$

where i is the index in one-dimensional array, $(i_0, i_1, \dots, i_{N-1})$ is the N-dimensional index, and $n_0, n_1, \dots, n_{N-1}$ represents the size of N-dimensional array.

1.2.2 Local Vertex Index

We represent local vertex index with the new notation. For a given vertex with local index $i \in id_v$ associated with a cell with global index $I \in ID_C$, we define an index convention as

$$a(a_0, a_1)_{2,2}, \quad (1.6)$$

where $a_0, a_1 \in \{0, 1\}$. We found the next point simply by adding 1 to the one-dimensional index as

$$b \equiv a + 1 \pmod{4}, \quad (1.7)$$

where the direction is naturally defined by pointing from vertex v_a to v_b .

1.2.3 Local Global Index convention

Assuming local vertex and edge number being counted in the same way as indicated in the mesh ??, where the counting starts from left bottom corner. We define the local to global mapping with the local index of vertex and global index of cell. Consider mapping the local index i of vertex v_i in the cell $C_{K(I_0, I_1)_{M, N}}$ as

$$id_v \ni i(i_0, i_1)_{m, n} \rightarrow I(I_0, I_1)_{M+1, N+1} \in ID_V. \quad (1.8)$$

We write the mapped global index with the reshape notation (1.5) as

$$ID_V \ni I(I_0 + i_0, I_1 + i_1)_{M+1, N+1}. \quad (1.9)$$

Note that the bottom global index of left corner vertex share the same N-dimensional index as the global index of the cell. We then discuss the local to global mapping for vertical and horizontal edges. Consider the starting vertex $id_v \ni i(i_0, i_1)$ of the edge e_i such that

- Horizontal Edge;

$$ID_E \ni I(I_0, I_1 + i_1)_{M, N+1}. \quad (1.10)$$

- Vertical Edge.

$$ID_E \ni I(I_0 + i_0, I_1)_{M+1, N} + M \times (N + 1). \quad (1.11)$$

1.3 Dofs and MFEM assembly

In FEM and FVM computation, we need to associate geometrical objects with degrees of freedom. For FVM scheme, the dofs associated with edge flux are straightforward. However, the MFEM dofs require careful treatment, especially for those dofs on the boundary. We follow the idea in Hughes (1987), we need three data processing arrays: ID, IEN and LM arrays. We start with the introduction of these three arrays as

- IEN array.

$$\text{IEN}(a, e) = A, \quad (1.12)$$

where a is the local dof index, e is the global element number and A is the global dof index.

- ID array.

$$\text{ID}(A) = \begin{cases} P, & A \notin \text{Essen}, \\ 0, & A \in \text{Essen}, \end{cases} \quad (1.13)$$

where P is the position used in global matrix assembly, and Essen denotes the boundary assigned with essential boundary condition.

- LM array.

$$\text{LM} = \text{ID}(\text{IEN}(a, e)). \quad (1.14)$$

In actual code implementation, we create LM array by screening all dofs prior to computation, while IEN and ID array are not actually stored.

1.3.1 Index Convention Example

For $H(\text{div})$ element, the dofs are mainly associated with edge like FVM, which is easy to comprehend and implement. Therefore, we chose to give a detailed description of how Modified Bernardi-Raugel is implemented with the concept of ID, IEN and LM arrays, where vertex dofs and edge dofs are mixed. In Figure 1.2, we illustrate

dofs in a bernardi regaue element, where two dofs (x dof and y dof) are associated with vertex nodes and one dof (flux dof) associated with edge nodes. We count x dof first y dof second, flux acorss horizontal edges third and the flux across vertical edges.

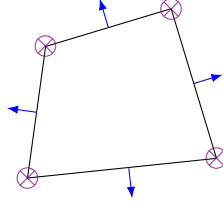


Figure 1.2: An exhibition of dofs associated with a modified Bernardi-Regual element.

In Figure 1.3, we show a simple 4×4 mesh with corresponding index associated with each dof marked. The total dof counts equal to $2 \times (3 \times 3) + 2 \times 3 + 3 \times 2 = 30$. For a column upwelling problem, We assume the following boundary condition is prescribed.

- Bottom Edge;
 - x dof fixed with zero;
 - y dof fixed with a constant upwelling speed;
 - normal flux dof fixed with a constant influx.
- Left and Right side Edges;
 - x dof fixed with zero;
 - y dof is free, given natural boundary condition (zero stress);
 - normal flux dof fixed with a zero flux.
- Top Edge;
 - x dof fixed with zero;

- y dof is free, given natural boundary condition (zero stress);
- normal flux dof is free.

We mark all the dof index that are not prescribed, i.e., will be determined by solving a linear system with superscript a *. These "free" dofs consist of interior dofs and boundary dofs that are given natural boundary condition. We can form a

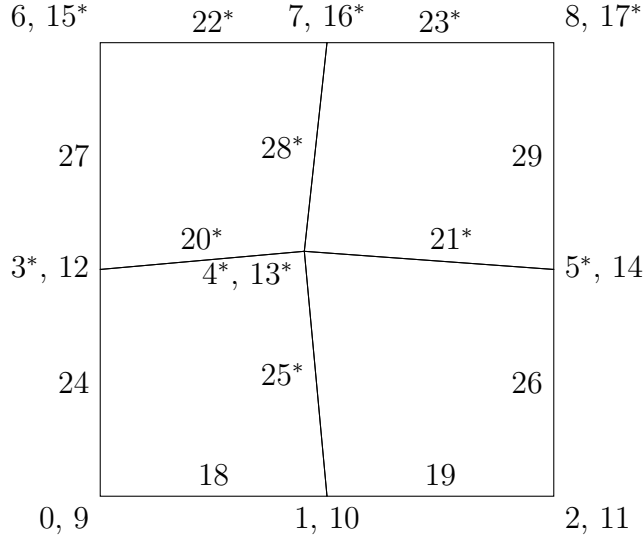


Figure 1.3: An exhibition of br dofs index with a 4×4 mesh.

reduced ID array with the information in Figure 1.3 as The reduced linear system

P	0	1	2	3	4	5	6	7	8	9	10	11
A	3	4	4	13	14	16	20	21	22	23	25	28

Table 1.1: Generated ID array relating to Figure 1.3.

excluding essential boundary dofs will be 12×12 .

1.4 Parallel Implementation

In this section, we discuss some ideas regarding parallel implementation of MFEM and FVM solvers, and present some preliminary timing results for each solver. The parallel implementation of the MFEM and FVM need different treatments.

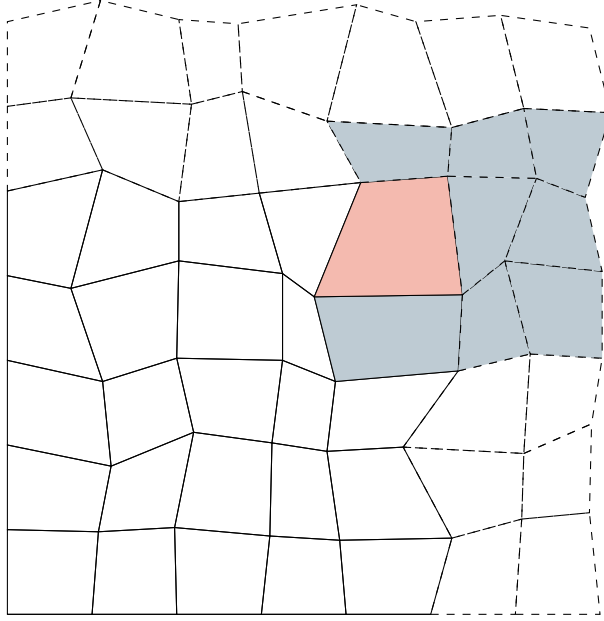


Figure 1.4: An exhibition of ghost region in parallel implementation of ML-WENO scheme.

For FVM discretization, the high order stencils posed some difficulties on parallel implementation. We need to extend from one processor to the adjacent processor with the so-called ghost region, which results in more communication during computation. Consider a ML-WENO(3,2) scheme, as shown in Figure 1.4, we show distributed ghost region with dashed lines. We need to determine the flux across the edge of the red cell. Therefore, a maximum 3×3 stencil should be attached to the outside cell. Hence, in a ML-WENO(3,2) scheme, a two-cell ghost layer should be distributed and updated everytime computing flux.

For MFEM discretization, we encounter different problems. The MFEM discretization, unlike FVM scheme, forms a sparse matrix. The sparse matrix are split across the processors. The dof in solution vector should be distributed accordingly. Therefore, everytime the solution is being reconstructed, communication between processors will be induced.

We provide strong scaling plot for our test parallel running. We run our MFEM

and FVM code with 1,2,4,6 processors

1.4.1 Parallel Index Convention

Index convention in parallel computing requires another "local" index kept within each processor. We can bypass such difficulty by using Hash table data structure that directly mapping to global index with out adding another "local" index. However, in the worst case Hash table suffers $O(n)$ time complexity in searching.

1.5 PETSc Implementation

PETSc, the Protable, Extensible Toolkit for Scientific Computation has been gaining popularity in scientific applications modelled by partial differential equations Balay et al. (2025b) and Balay et al. (2025a). We used three main features from PETSc package.

- Vector (Vec) and matrix (Mat) data structure.
- Data management object (DM).
- Standard linear solver (KSP).

We store global data in Vec and Mat data structure provided by PETSc. Vec and Mat objects in PETSc can complete simple add, dot operations without distinguish between sequential or parallel.

Note that PETSc provides a built-in data management object DMStag. However, we use a more generic data management object DMDA for structured mesh because we formed reduced linear system in MFEM computation. DMStag, on the other hand, does not provide a straightforward function that can create reduced system for us.

We use the built in KSP solver, e.g., KSPMINRES and KSPCG solver for our application when we solve large sparse system. However, for small system such as

4×4 system solved in ML-WENO stencil polynomial computation, we call lapack directly. We also use provided preconditioner.

Works Cited

Satish Balay, Shrirang Abhyankar, Mark F. Adams, Steven Benson, Jed Brown, Peter Brune, Kris Buschelman, Emil Constantinescu, Lisandro Dalcin, Alp Dener, Victor Eijkhout, Jacob Faibussowitsch, William D. Gropp, Václav Hapla, Tobin Isaac, Pierre Jolivet, Dmitry Karpeev, Dinesh Kaushik, Matthew G. Knepley, Fande Kong, Scott Kruger, Dave A. May, Lois Curfman McInnes, Richard Tran Mills, Lawrence Mitchell, Todd Munson, Jose E. Roman, Karl Rupp, Patrick Sanan, Jason Sarich, Barry F. Smith, Hansol Suh, Stefano Zampini, Hong Zhang, Hong Zhang, and Junchao Zhang. PETSc/TAO users manual. Technical Report ANL-21/39 - Revision 3.23, Argonne National Laboratory, 2025a.

Satish Balay, Shrirang Abhyankar, Mark F. Adams, Steven Benson, Jed Brown, Peter Brune, Kris Buschelman, Emil M. Constantinescu, Lisandro Dalcin, Alp Dener, Victor Eijkhout, Jacob Faibussowitsch, William D. Gropp, Václav Hapla, Tobin Isaac, Pierre Jolivet, Dmitry Karpeev, Dinesh Kaushik, Matthew G. Knepley, Fande Kong, Scott Kruger, Dave A. May, Lois Curfman McInnes, Richard Tran Mills, Lawrence Mitchell, Todd Munson, Jose E. Roman, Karl Rupp, Patrick Sanan, Jason Sarich, Barry F. Smith, Stefano Zampini, Hong Zhang, Hong Zhang, and Junchao Zhang. PETSc Web page. <https://petsc.org/>, 2025b. URL <https://petsc.org/>.

Bruce G. Baumgart. A polyhedron representation for computer vision. In *Proceedings of the May 19-22, 1975, National Computer Conference and Exposition, AFIPS '75*, page 589–596, New York, NY, USA, 1975. Association for Computing Machinery. ISBN 9781450379199. doi: 10.1145/1499949.1500071. URL <https://doi.org/10.1145/1499949.1500071>.

Thomas J.R. Hughes. *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Dover Publications, 1987. ISBN 9780486411811.